

MethylPipeline User Manual

Operational runbook for package usage and staged workflows

MethylPipeline Team

2026-05-21

Table of contents

1	MethylPipeline User Manual	6
1.1	What this manual is	6
1.2	Reading paths	6
1.3	End-to-end workflow map	6
1.4	Scope boundaries	7
1.5	Canonical references	7
I	Part I: Setup and Package Orientation	8
2	Prerequisites and Environment	9
2.1	Purpose	9
2.2	Required tools	9
2.3	Environment bootstrap	9
2.4	Verify CLI availability	9
2.5	Runtime assumptions	10
2.6	Success checks	10
2.7	Common failures and fixes	10
2.8	See also	10
3	Project Config and Layout	11
3.1	Purpose	11
3.2	Minimum structure	11
3.3	Validation stage keys that matter most	11
3.4	Output layout	12
3.5	Handoff model	12
3.6	Do not do this	12
3.7	Success checks	13
3.8	See also	13
4	Package Reference (Individual Usage)	14
4.1	Purpose	14
4.2	Package quick map	14
4.3	Standalone examples	15
4.3.1	Alignment QC	15
4.3.2	Enrichment from mapper output	15
4.3.3	Blind prediction with frozen model	15
4.4	Orchestrated equivalents in <code>methy1-validation</code>	15
4.5	Integration diagram	15
4.6	Do not do this	16

4.7	See also	16
II	Part II: Staged Solution Workflow	17
5	Stage: Stability	18
5.1	Purpose	18
5.2	Entry command	18
5.3	Required config keys	18
5.4	Process model	19
5.5	Expected outputs	19
5.6	Success checks	19
5.7	Do not do this	19
5.8	Recovery	19
5.9	See also	20
6	Stage: Freeze	21
6.1	Purpose	21
6.2	Entry command	21
6.3	Prerequisites	21
6.4	Process model	21
6.5	Expected outputs	22
6.6	Success checks	22
6.7	Recommended post-freeze gate	22
6.8	Do not do this	22
6.9	Recovery	22
6.10	See also	23
7	Stage: Model	24
7.1	Purpose	24
7.2	Common execution patterns	24
7.2.1	Pattern A: Direct production model build	24
7.2.2	Pattern B: Backend model-MC comparison then select best	24
7.3	Backend behavior	24
7.4	Process model	25
7.5	Required config keys	25
7.6	Expected outputs	26
7.7	Success checks	26
7.8	Do not do this	26
7.9	See also	26
8	Stage: Post-model Validation	27
8.1	Purpose	27
8.2	Entry command	27
8.3	Prerequisites	27
8.4	Process model	28
8.5	Expected outputs	28
8.6	Success checks	28

8.7	Do not do this	29
8.8	Recovery	29
8.9	See also	29
9	Stage: Blind Prediction	30
9.1	Purpose	30
9.2	Important boundary	30
9.3	Entry command (standalone)	30
9.4	Process model	30
9.5	Required config pattern	30
9.6	Expected outputs	31
9.7	Success checks	31
9.8	Do not do this	31
9.9	Recovery	31
9.10	See also	31
III	Part III: Operations and Reliability	32
10	Artifacts and QA Checks	33
10.1	Purpose	33
10.2	Artifact map	33
10.3	Handoff integrity checks	34
10.4	QA checklist per stage	34
10.4.1	Stability	34
10.4.2	Freeze	34
10.4.3	Model	34
10.4.4	Post-model validation	34
10.4.5	Blind prediction	35
10.5	Escalation triggers	35
10.6	See also	35
11	Troubleshooting and Recovery	36
11.1	Purpose	36
11.2	Failure routing	36
11.3	Common problems	36
11.4	Safe recovery sequence	37
11.5	Resume examples	37
11.6	Operational guardrails	37
11.7	See also	37
12	Command Cookbook	38
12.1	Purpose	38
12.2	1) Full staged run (strict profile example)	38
12.3	2) Backend comparison and selection	38
12.4	3) Rollout comparison report	38
12.5	4) Standalone enricher run from mapper output	39
12.6	5) Standalone blind prediction	39

12.7	6) Manual rendering commands (this manual)	39
12.8	7) Distributed queue (one task)	39
12.9	8) Hyperparameter search grid (example)	39
12.10	Execution notes	40
13	Distributed MethyIValidation on shared storage	41
13.1	Purpose	41
13.2	Prerequisites	41
13.3	Layout (under <code>monte_carlo_runs/</code>)	41
13.4	Command sequence	42
13.5	Example paths (<code>/work</code> pattern)	42
13.6	Recovery	42
13.7	See also	43
14	Optional hyperparameter search	44
14.1	When to use	44
14.2	What it is	44
14.3	Command example	44
14.4	Pitfalls	45
14.5	See also	45

1 MethylPipeline User Manual

1.1 What this manual is

This is the operational companion to the theory book. It is designed for execution teams who need:

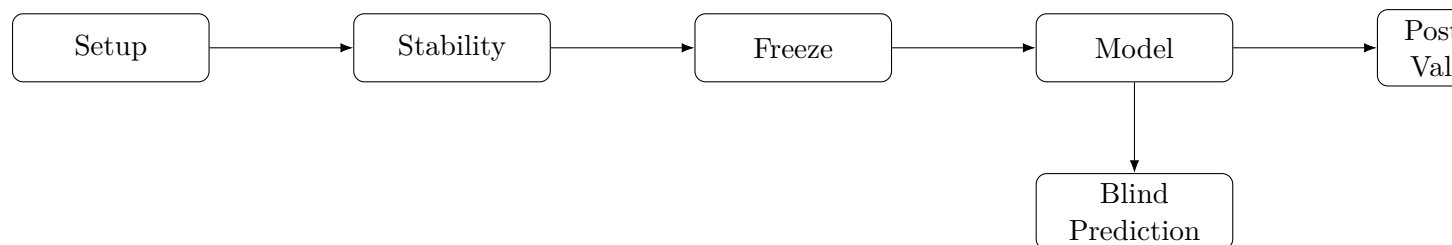
- exact commands,
- required config keys,
- expected output artifacts,
- stage handoff checks,
- recovery actions when a run fails.

For statistical background, use `docs/theory/`.

1.2 Reading paths

- **Fast path (stage execution):** Chapters 04 to 08.
- **New project setup:** Chapters 01, 02, then 04.
- **Daily operations:** Chapters 09 to 11.
- **Scaling MC across machines / optional hyperparameter tuning:** Chapters 12 and 13 (shared `/work` storage, queue subcommands, and the `methyl-hyperparam-search` grid driver).
- **Standalone package usage:** Chapter 03.

1.3 End-to-end workflow map



1.4 Scope boundaries

- `methyl-validation` orchestrates stability, freeze, model, and post-model validation.
- Blind-only prediction is documented as a standalone `methyl-predictor` operation.
- Package behavior is documented from implementation in `packages/*`.

1.5 Canonical references

- Workflow reference: `docs/WORKFLOW_DIAGRAM_PACK_Healthy_vs_PCa1-4-CG.md`
- Validation orchestration: `packages/methylvalidation/docs/USAGE.md`
- Distributed queue (shared storage, `plan-runs` / `run-task` / `aggregate-results`):
`packages/methylvalidation/docs/DISTRIBUTED_QUEUE.md`
- Hyperparameter search (objective (J), `methyl-hyperparam-search`): `packages/methylvalidation/docs/HYPERPARAM_SEARCH.md`
- Validation flags and config model:
 - `packages/methylvalidation/methyl_validation/cli.py`
 - `packages/methylvalidation/methyl_validation/config.py`

Part I

Part I: Setup and Package Orientation

2 Prerequisites and Environment

2.1 Purpose

Set up a reproducible execution environment for all package CLIs and workflow stages.

2.2 Required tools

- Python environment in repo-local `.venv`
- Installed package CLIs (`methyl-*`)
- Quarto for manual rendering (optional for pipeline runs)
- TeX engine for PDF rendering (`quarto install tinytex` recommended)

2.3 Environment bootstrap

From repo root:

```
python3.12 -m venv .venv
source .venv/bin/activate
bash scripts/install_all.sh
```

Optional dependency sets:

```
bash scripts/install_all.sh --pipeline-reqs
bash scripts/install_all.sh --gpu-reqs
```

2.4 Verify CLI availability

```
source .venv/bin/activate
methyl-validation --help
methyl-centroid --help
methyl-detector --help
methyl-mapper --help
methyl-enricher --help
methyl-predictor --help
```

2.5 Runtime assumptions

- Activate `.venv` before every run.
- Use absolute paths in project configs where possible.
- Keep `samples_base_path` consistent across stages.

2.6 Success checks

- `.venv` activates without import errors.
- `methylation-validation --help` prints options.
- Project JSON path resolves on the execution host.

2.7 Common failures and fixes

- **command not found:** activate `.venv` and reinstall with `scripts/install_all.sh`.
- **module import errors:** reinstall editable packages from repo root.
- **path mismatch between hosts:** set `step_config.validation.path_remap` or `--path-remap OLD=NEW`.

2.8 See also

- `README.md`
- `docs/DEPLOYMENT.md`

3 Project Config and Layout

3.1 Purpose

Define a project JSON that can run through stability, freeze, model, and evaluation without per-stage schema drift.

3.2 Minimum structure

Use one project file containing:

- global cohort metadata (`controls`, `diseases`, `comparisons`),
- filesystem anchors (`output_base`, `samples_base_path`),
- step-specific overrides under `step_config`.

Core stage driver:

- `step_config.validation` (parsed into `MonteCarloConfig` by `methy1-validation`).

3.3 Validation stage keys that matter most

- split policy: `train_fraction`, `n_iterations`, `seed`
- stability controls:
 - `run_stability`
 - `stability_dmp_freq`
 - `stability_min_balanced_accuracy`
 - `stability_gene_freq`
 - `stability_featurecuts_enabled`
 - `stability_target_balanced_accuracy`
 - `stability_min_selected_dmps`
- freeze/model paths:
 - `freeze_stable_dmp_csv` (optional)
 - `production_output_dir` (optional)
- backend/model:
 - `model_backend`
 - `feature_mode`
 - `ecdf_second_stage_enabled`
 - backend-specific `tabular_*` / `generative_*` keys

3.4 Output layout

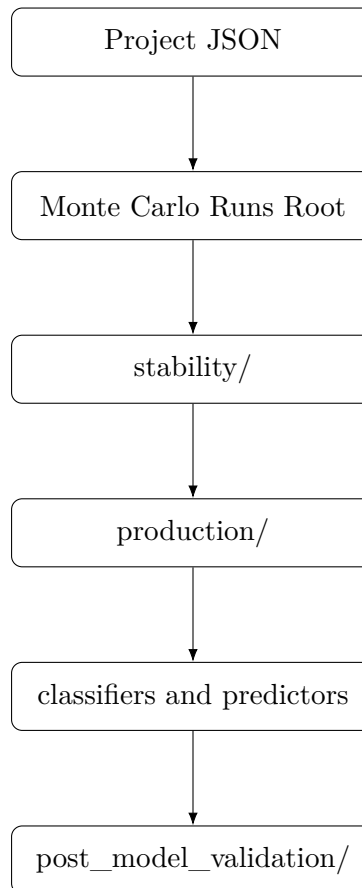
Most runs write to:

`<output_base>/<project_name>/monte_carlo_runs/`

Important subtrees:

- `run_XXXX/` (per-iteration artifacts)
- `stability/` (stable panel and summaries)
- `production/` (freeze and final model outputs)
- `model_mc/` (backend comparison runs)
- `post_model_validation/` (frozen-model descriptive evaluation)

3.5 Handoff model



3.6 Do not do this

- Do not put blind-only predictor config (`predictor.blind`) into validation-driven runs.
- Do not change `samples_base_path` between stages without explicit remap.

- Do not run `--freeze` before stable panel exists.

3.7 Success checks

- Project loads in `methyl-validation --project ....`
- `step_config.validation` exists and parses.
- Run directories appear under expected root.

3.8 See also

- `docs/theory/chapters/11-project-configuration.qmd`
- `docs/theory/chapters/13-configuration-reference.qmd`

4 Package Reference (Individual Usage)

4.1 Purpose

Summarize each active package as a standalone tool, then map it to staged `methyl-validation` orchestration.

4.2 Package quick map

Package	CLI	Primary role	Typical standalone output
<code>methylalignmentqc</code>	<code>methyl-qc</code>	Parse alignment/dup metrics into normalized JSON	<code>alignment_qc/<sample>.json</code>
<code>methylcentroid</code>	<code>methyl-centroid</code>	Build cohort centroids from methylation files	centroid artifacts per group
<code>methyldetector</code>	<code>methyl-detector</code>	Discover/classify DMPs	<code>dmps-*-discovery.csv</code> , <code>result*.json</code>
<code>methylmapper</code>	<code>methyl-mapper</code>	Map DMP loci to genes/features	<code>all-gene_name-combined.csv</code>
<code>methylenricher</code>	<code>methyl-enricher</code>	Enrichment/module interpretation	<code>enrichment_merged.csv</code> , <code>modules_ranked.csv</code>
<code>methylclassifier</code>	<code>methyl-classifier</code>	Train production classifier (ECDF path)	<code>classifiers/*.pkl</code>
<code>methylpredictor</code>	<code>methyl-predictor</code>	Predict and report metrics/inference	<code>predictors/*/validation_metrics</code> or blind reports
<code>methylvalidation</code>	<code>methyl-validation</code>	Orchestrate MC/stability/freeze/model workflows	<code>monte_carlo_runs/*</code>
<code>methyl-disease-progression</code>	<code>methyl-disease-progression</code>	Cross-stage synthesis from mapper/enricher outputs	<code>progression/*.csv</code> , <code>summary.json</code>

4.3 Standalone examples

4.3.1 Alignment QC

```
source .venv/bin/activate
methyl-qc --metrics-root /data/metrics --output-dir /work/alignment_qc
```

4.3.2 Enrichment from mapper output

```
source .venv/bin/activate
methyl-enricher --input /work/project/mapper/pca_pca1/all-gene_name-combined.csv --modules --o
```

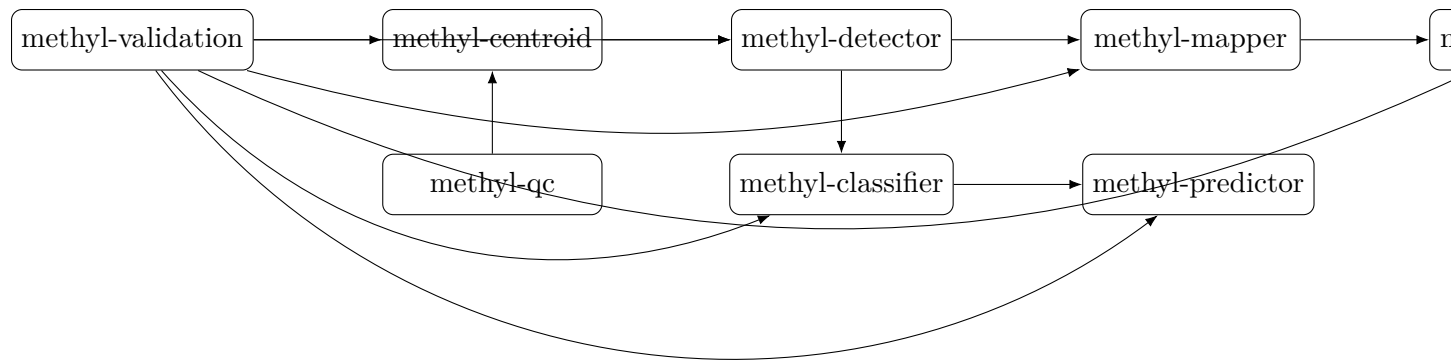
4.3.3 Blind prediction with frozen model

```
source .venv/bin/activate
methyl-predictor --project /work/project/monte_carlo_runs/production/project.json
```

4.4 Orchestrated equivalents in methyl-validation

- Stability stage runs centroid+detector per iteration.
- Freeze stage runs centroid->detector->mapper->enricher (and optional progression).
- Model stage runs:
 - ecdf: classifier->predictor
 - tabular_sklearn / generative_hybrid: in-process backend train/predict
- Post-model validation runs frozen-model descriptive holdouts.

4.5 Integration diagram



4.6 Do not do this

- Do not assume standalone package defaults match project-orchestrated overrides.
- Do not skip `project.json` artifact handoff checks when chaining packages manually.

4.7 See also

- `README.md`
- `packages/*/docs/USAGE.md`

Part II

Part II: Staged Solution Workflow

5 Stage: Stability

5.1 Purpose

Run repeated train/validation iterations to identify DMPs that recur across runs and materialize a stable production panel.

5.2 Entry command

```
source .venv/bin/activate
methyl-validation --project /work/prostate-cancer/configs/project_Healthy_vs_PCa1-4-CG.json --
```

Optional strict FeatureCuts alignment:

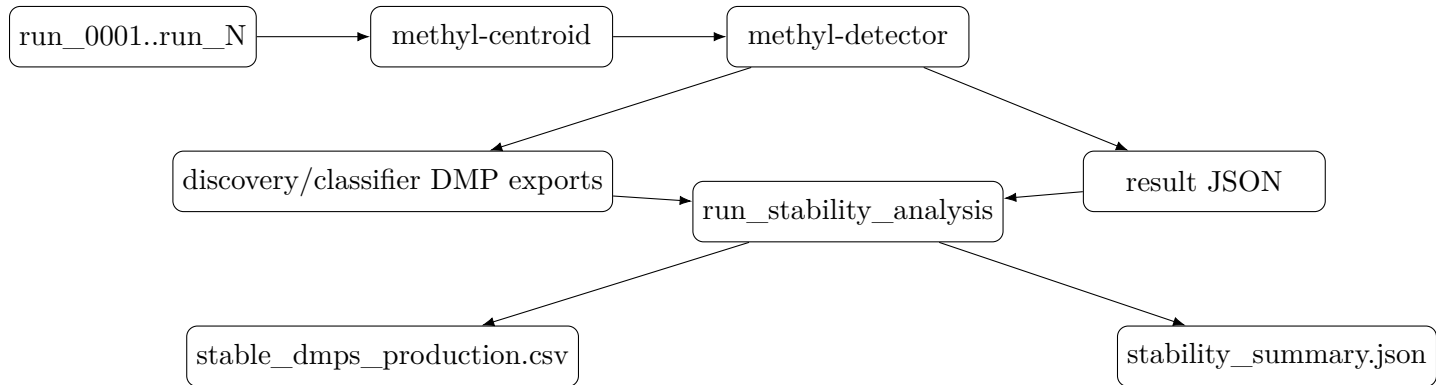
```
source .venv/bin/activate
methyl-validation --project /work/prostate-cancer/configs/project_Healthy_vs_PCa1-4-CG.json \
  --stability --stability-featurecuts --stability-target-ba 0.95 --stability-min-selected-dmps
```

5.3 Required config keys

Under `step_config.validation`:

- `n_iterations`, `train_fraction`, `seed`
- `run_stability`
- `stability_dmp_freq`
- `stability_min_balanced_accuracy`
- `stability_gene_freq`
- optional FeatureCuts controls:
 - `stability_featurecuts_enabled`
 - `stability_target_balanced_accuracy`
 - `stability_min_selected_dmps`

5.4 Process model



5.5 Expected outputs

- <mc_root>/stability/stable_dmps_production.csv
- <mc_root>/stability/stability_summary.json
- <mc_root>/all_metrics.csv
- <mc_root>/metrics_summary.json
- <mc_root>/baseline_manifest.json

Where <mc_root> is: <output_base>/<project_name>/monte_carlo_runs/

5.6 Success checks

- stable_dmps_production.csv exists and is non-empty.
- all_metrics.csv has successful rows near expected run count.
- stability_summary.json reports configured thresholds.

5.7 Do not do this

- Do not proceed to freeze when stable panel is missing or empty.
- Do not interpret blind outputs as stability quality evidence.

5.8 Recovery

- Resume interrupted run:

```
source .venv/bin/activate
methyl-validation --project /work/prostate-cancer/configs/project_Healthy_vs_PCa1-4-CG.json --
```

- If panel is too small: lower `stability_dmp_freq` or increase `n_iterations`.
- If run quality is too low: tune detector and/or adjust `stability_min_balanced_accuracy`.

5.9 See also

- `docs/theory/chapters/12-two-workflows.qmd`
- `docs/theory/chapters/15-model-creation-and-validation.qmd`
- If many MC iterations are split across several hosts, use shared `output_base` and the distributed queue flow: [Distributed MethylValidation on shared storage](#) (12).

6 Stage: Freeze

6.1 Purpose

Create frozen production artifacts using the stable panel and generate biological interpretation outputs (mapper/enricher, optional progression).

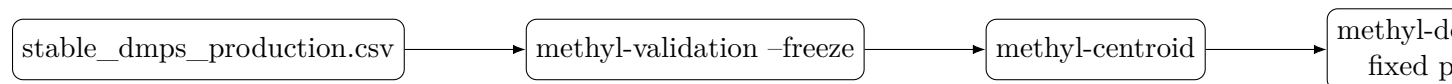
6.2 Entry command

```
source .venv/bin/activate
methyl-validation --project /work/prostate-cancer/configs/project_Healthy_vs_PCa1-4-CG.json --
```

6.3 Prerequisites

- Stability stage completed.
- Stable panel available at:
 - configured `step_config.validation.freeze_stable_dmp_csv`, or
 - default `<mc_root>/stability/stable_dmps_production.csv`.

6.4 Process model



6.5 Expected outputs

- `<mc_root>/production/project.json`
- `<mc_root>/production/production_summary.json`
- mapper outputs under `production/mapper/...`
- enricher outputs under `production/enricher/...`
- optional progression outputs under `production/progression/...`

6.6 Success checks

- `production/project.json` exists and includes fixed panel usage for detector.
- expected mapper and enricher files are present.
- stage-specific logs show successful return codes.

6.7 Recommended post-freeze gate

Run biological readiness before model training:

```
source .venv/bin/activate
methyl-validation biological-readiness /work/prostate-cancer/Healthy_vs_PCa1-4-CG
```

This chained gate verifies:

- enricher completeness (`verify-complete`),
- strict progression input completeness (`--strict-missing`),
- stability/freeze/progression readiness verdict.

To include advisory Grok commentary in the readiness section:

```
methyl-validation biological-readiness /work/prostate-cancer/Healthy_vs_PCa1-4-CG --grok-review
```

Without `--grok-review`, the chain keeps Grok disabled by default and still enforces deterministic checks.

6.8 Do not do this

- Do not run freeze before stability panel exists.
- Do not delete `production/project.json`; later model and predictor stages depend on it.

6.9 Recovery

- If freeze fails because stable panel missing, rerun stability or set `freeze_stable_dmp_csv`.
- If path mismatch occurs across machines, use `path_remap` or `--path-remap`.

6.10 See also

- `packages/methylvalidation/docs/USAGE.md`
- `docs/theory/chapters/14-user-guide.qmd`

7 Stage: Model

7.1 Purpose

Train and select final production model artifacts after freeze and biological-readiness checks.

7.2 Common execution patterns

7.2.1 Pattern A: Direct production model build

```
source .venv/bin/activate
methyl-validation biological-readiness /work/prostate-cancer/Healthy_vs_PCa1-4-CG
methyl-validation --project /work/prostate-cancer/configs/project_Healthy_vs_PCa1-4-CG.json --r
```

7.2.2 Pattern B: Backend model-MC comparison then select best

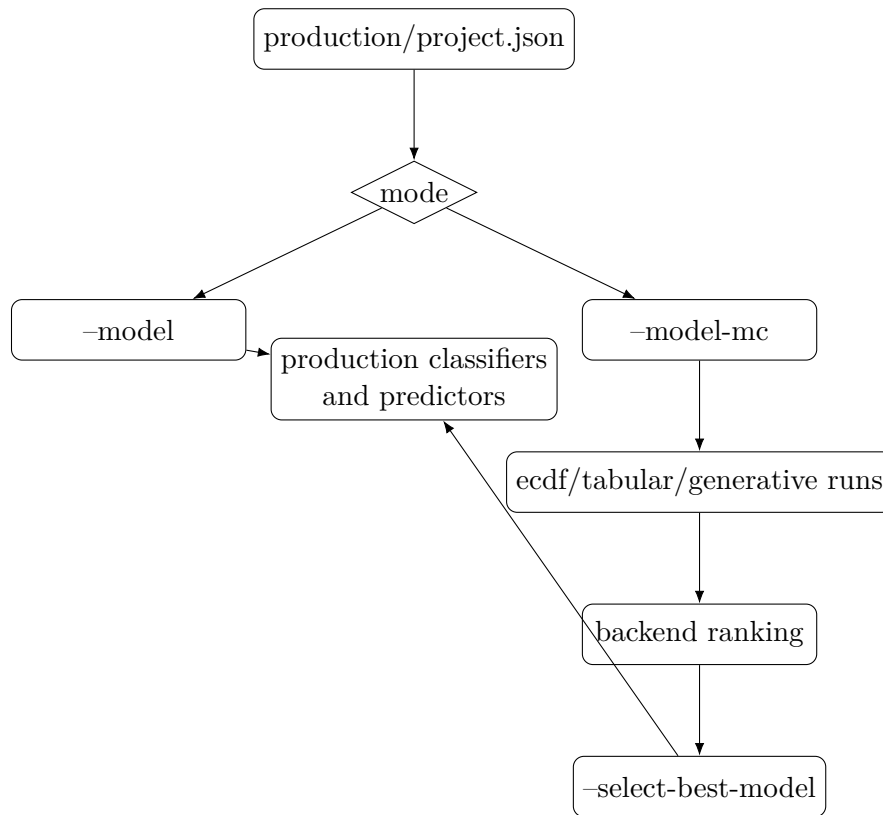
```
source .venv/bin/activate
methyl-validation --project /work/prostate-cancer/configs/project_Healthy_vs_PCa1-4-CG.json --r
methyl-validation --project /work/prostate-cancer/configs/project_Healthy_vs_PCa1-4-CG.json \
  --select-best-model --model-mc-all --selection-metric balanced_accuracy --selection-stat med
```

7.3 Backend behavior

- ecdf: methyl-classifier -> methyl-predictor
- tabular_sklearn: in-process bundle -> train -> predict
- generative_hybrid: in-process bundle -> train -> predict

Configured by `step_config.validation.backend_profiles` (plus optional CLI backend override).

7.4 Process model



7.5 Required config keys

- backend_profiles
- backend-specific params under each profile (for example `feature_mode`, `ecdf_second_stage_enabled`, `tabular_methods`, `generative_latent_dim`)
- tabular controls:
 - `tabular_methods`
 - `tabular_method_selection_metric`
- generative controls:
 - `generative_latent_dim`, `generative_epochs`, and related keys
- optional gate:
 - `require_biological_review_for_model`
 - `biological_review_confirmed`

Legacy flat backend keys are rejected. Migrate older configs with:

```
methyl-validation-migrate-backend-config /path/to/project.json --in-place
```

7.6 Expected outputs

- `<mc_root>/production/classifiers/...`
- `<mc_root>/production/predictors/...`
- if model-MC:
 - `<mc_root>/model_mc/shared/run_XXXX/...`
 - `<mc_root>/model_mc/<backend>/all_metrics.csv`
 - `<mc_root>/model_mc/backend_ranking.csv`
 - `<mc_root>/production/selected_backend.json` (after selection)

7.7 Success checks

- final classifier/predictor artifacts exist in production root.
- selected backend metadata exists when using selection workflow.
- metrics summaries include expected selection metric fields.
- the last biological-readiness report is `go/go_with_risks` and reviewed by humans.

7.8 Do not do this

- Do not run model before freeze.
- Do not skip biological readiness review when production artifacts changed.
- Do not claim backend superiority without model-MC summaries.

7.9 See also

- `docs/theory/chapters/15-model-creation-and-validation.qmd`
- `packages/methylvalidation/docs/ROLLOUT.md`

8 Stage: Post-model Validation

8.1 Purpose

Measure descriptive performance distributions using frozen production artifacts without retraining.

8.2 Entry command

```
source .venv/bin/activate
methyl-validation --project /work/prostate-cancer/configs/project_Healthy_vs_PCa1-4-CG.json --]
```

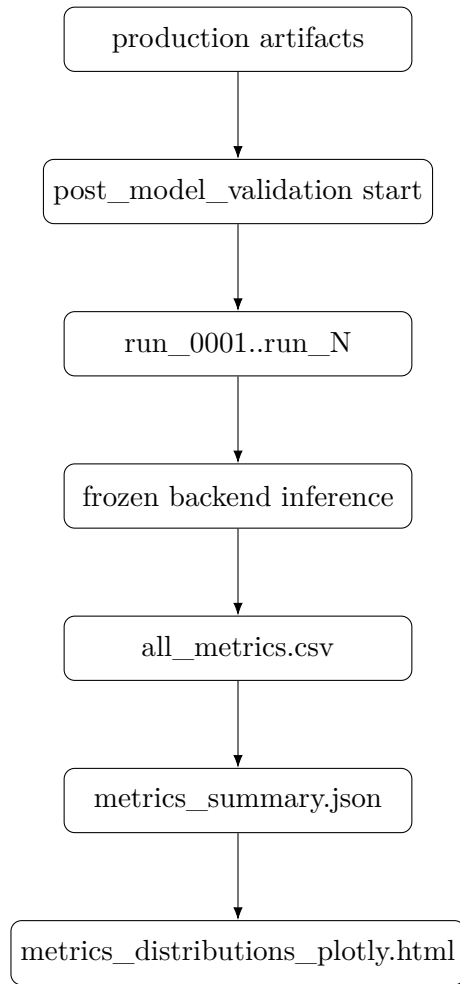
Optional backend override:

```
source .venv/bin/activate
methyl-validation --project /work/prostate-cancer/configs/project_Healthy_vs_PCa1-4-CG.json \
  --post-model-validation --post-model-backend ecdf
```

8.3 Prerequisites

- Freeze/model artifacts already built in production root.
- Valid production/project.json.

8.4 Process model



8.5 Expected outputs

- `<mc_root>/post_model_validation/run_XXXX/...`
- `<mc_root>/post_model_validation/all_metrics.csv`
- `<mc_root>/post_model_validation/metrics_summary.json`
- `<mc_root>/post_model_validation/metrics_distributions_plotly.html`

8.6 Success checks

- run directories created under `post_model_validation`.
- metrics summary includes key metrics (`balanced_accuracy`, `macro_f1`, plus available calibration metrics).
- distribution plot generated.

8.7 Do not do this

- Do not treat post-model validation as model training.
- Do not compare candidate and baseline manually without fixed thresholds; use rollout comparison when promoting.

8.8 Recovery

- Resume interrupted post-model validation:

```
source .venv/bin/activate
methyl-validation --project /work/prostate-cancer/configs/project_Healthy_vs_PCa1-4-CG.json \
  --post-model-validation --resume
```

8.9 See also

- `packages/methylvalidation/docs/USAGE.md`
- `packages/methylvalidation/docs/ROLLOUT.md`

9 Stage: Blind Prediction

9.1 Purpose

Run inference on newly arrived unlabeled samples using frozen model artifacts.

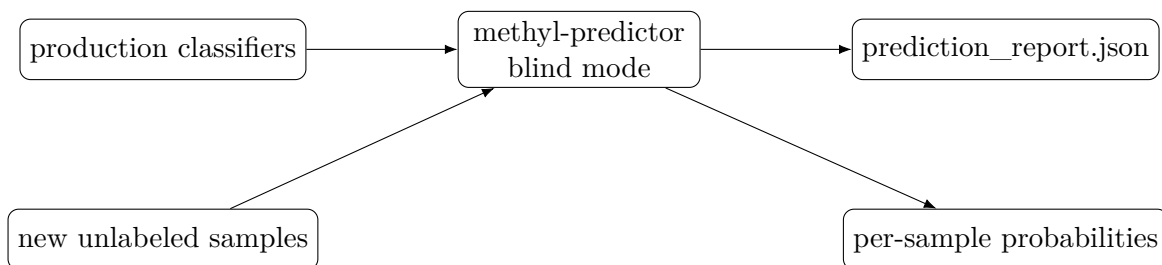
9.2 Important boundary

Blind-only prediction is a standalone `methyl-predictor` operation. It is intentionally outside `methyl-validation` accuracy workflows.

9.3 Entry command (standalone)

```
source .venv/bin/activate
methyl-predictor --project /work/prostate-cancer/Healthy_vs_PCa1-4-CG/monte_carlo_runs/product.
```

9.4 Process model



9.5 Required config pattern

`step_config.predictor` should reference blind/test-blind inputs and model paths resolved from frozen production project.

9.6 Expected outputs

- blind prediction report JSON
- per-sample class probabilities and predicted subgroup labels

9.7 Success checks

- predictor completes without missing model-path errors.
- expected blind report file exists.
- per-sample prediction rows match input sample count.

9.8 Do not do this

- Do not use blind-only prediction results as labeled validation accuracy claims.
- Do not run blind-only predictor settings through `methy1-validation` MC paths.

9.9 Recovery

- If model path resolution fails, confirm `production/project.json` and classifier artifacts exist.
- If sample-id join fails, verify sample path naming and predictor input lists.

9.10 See also

- `packages/methylpredictor/docs/USAGE.md`
- `docs/theory/chapters/14-user-guide.qmd`

Part III

Part III: Operations and Reliability

10 Artifacts and QA Checks

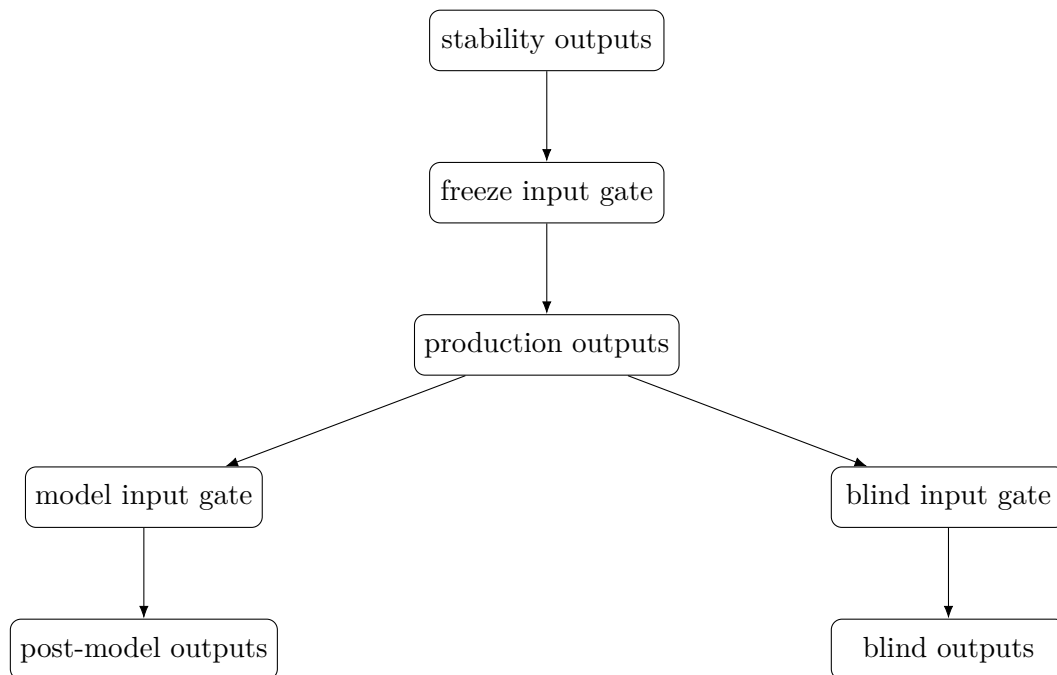
10.1 Purpose

Provide a single checklist for expected files at each stage and quick quality gates before proceeding.

10.2 Artifact map

Stage	Required artifacts	Gate to proceed
Stability	<code>stability/stable_dmps_production/</code> , <code>stability_summary.json</code> , <code>all_metrics.csv</code>	Stable panel exists and is non-empty
Freeze	<code>production/project.json</code> , mapper/enricher outputs	Production project exists; fixed-panel detector run completed
Model	<code>production/classifiers/*</code> , <code>production/classifiers/training_metrics.json</code> (all model backends), <code>production/predictors/*</code>	Final classifier/predictor artifacts present
Post-model validation	<code>post_model_validation/all_metrics_summary.json</code> , plotly report	Distribution summaries generated
Blind prediction	blind prediction report + per-sample probabilities	All input blind samples receive predictions

10.3 Handoff integrity checks



10.4 QA checklist per stage

10.4.1 Stability

- Iteration count in metrics roughly matches configured `n_iterations`.
- Stable panel size is consistent with selected thresholds.

10.4.2 Freeze

- `production/project.json` references fixed panel detector mode.
- Mapper/enricher outputs exist for each expected comparison group.

10.4.3 Model

- Backend used is recorded and consistent with config.
- If selection used, ranking artifacts exist and chosen backend is traceable.

10.4.4 Post-model validation

- Metrics summary includes required decision metrics.
- Distribution artifacts generated and readable.

10.4.5 Blind prediction

- Prediction outputs include all requested samples.
- Operational report does not include false accuracy claims.

10.5 Escalation triggers

- Empty stable panel.
- Missing production project.
- No model artifacts after model stage.
- Repeated run failures in same step across resumed runs.

10.6 See also

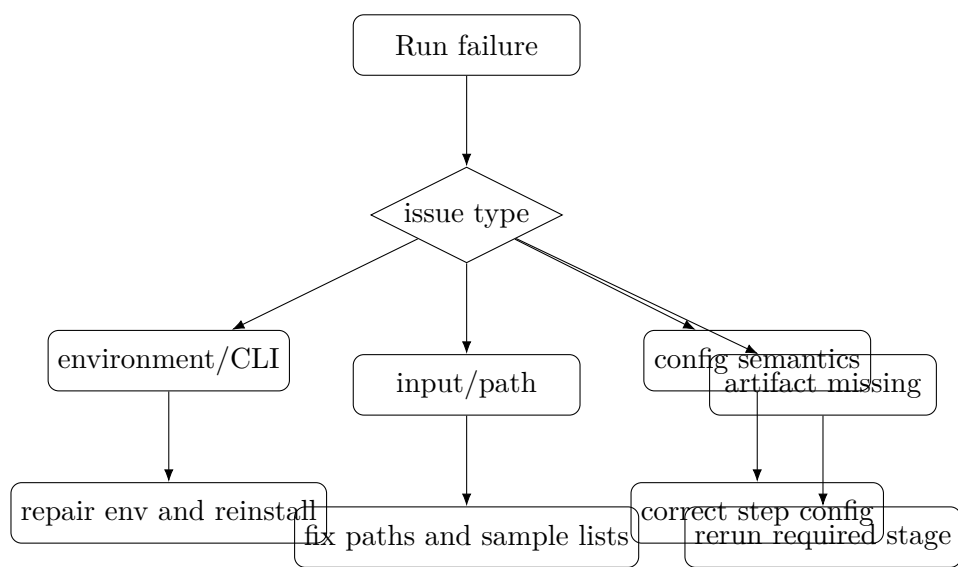
- `packages/methylvalidation/docs/USAGE.md`
- `packages/methylvalidation/docs/IMPLEMENTATION.md`

11 Troubleshooting and Recovery

11.1 Purpose

Standardize response for common stage failures and reduce ad-hoc debugging.

11.2 Failure routing



11.3 Common problems

Symptom	Likely cause	Recovery
command not found: methyl-validation	.venv inactive or install missing	Activate .venv, rerun scripts/install_all.sh
freeze fails for missing stable panel	stability not run or wrong path	Run --stability or set freeze_stable_dmp_csv
model fails due to biological review gate	require_biological_review_for and not confirmed	set model=true biological_review_confirmed=true after review
path file-not-found across hosts	environment path differences	use path_remap/--path-remap OLD=NEW

Symptom	Likely cause	Recovery
blind predictor rejected in validation mode	blind settings passed to <code>methyl-validation</code>	run blind mode with standalone <code>methyl-predictor</code>

11.4 Safe recovery sequence

1. Validate environment and CLI availability.
2. Confirm previous-stage artifacts exist.
3. Re-run failed stage only.
4. Use resume mode where supported.
5. Re-check QA gates before advancing.

11.5 Resume examples

```
source .venv/bin/activate
methyl-validation --project /work/prostate-cancer/configs/project_Healthy_vs_PCa1-4-CG.json --resume
methyl-validation --project /work/prostate-cancer/configs/project_Healthy_vs_PCa1-4-CG.json --resume
```

11.6 Operational guardrails

- Keep original run directories for traceability.
- Prefer additive reruns over manual artifact edits.
- Record config overrides used in reruns.

11.7 See also

- `docs/code_first_discovery_report.md`
- `docs/config_parameter_matrix.md`

12 Command Cookbook

12.1 Purpose

Provide copy-ready command sequences for the full solution lifecycle.

12.2 1) Full staged run (strict profile example)

```
source .venv/bin/activate
PROJECT="/work/prostate-cancer/configs/project_Healthy_vs_PCa1-4-CG.json"

methyl-validation --project "$PROJECT" --stability
methyl-validation --project "$PROJECT" --freeze
methyl-validation --project "$PROJECT" --model
methyl-validation --project "$PROJECT" --post-model-validation
```

12.3 2) Backend comparison and selection

```
source .venv/bin/activate
PROJECT="/work/prostate-cancer/configs/project_Healthy_vs_PCa1-4-CG.json"

methyl-validation --project "$PROJECT" --model-mc --model-mc-all
methyl-validation --project "$PROJECT" --select-best-model --model-mc-all \
  --selection-metric balanced_accuracy --selection-stat median
```

12.4 3) Rollout comparison report

```
source .venv/bin/activate
PROJECT="/work/prostate-cancer/configs/project_Healthy_vs_PCa1-4-CG.json"
BASE="/work/prostate-cancer/Healthy_vs_PCa1-4-CG/monte_carlo_runs/model_mc/ecdf/metrics_summary"
CAND="/work/prostate-cancer/Healthy_vs_PCa1-4-CG/monte_carlo_runs/model_mc/tabular_sklearn/metrics_summary"

methyl-validation --project "$PROJECT" --rollout-compare \
  --baseline-summary "$BASE" --candidate-summary "$CAND"
```

12.5 4) Standalone enricher run from mapper output

```
source .venv/bin/activate
methyl-enricher \
  --input /work/prostate-cancer/Healthy_vs_PCa1-4-CG/mapper/all/pca_pca1/all-gene_name-combined
  --modules \
  --library-preset cancer-extended \
  --outdir /work/prostate-cancer/Healthy_vs_PCa1-4-CG/enricher/all/pca_pca1
```

12.6 5) Standalone blind prediction

```
source .venv/bin/activate
methyl-predictor --project /work/prostate-cancer/Healthy_vs_PCa1-4-CG/monte_carlo_runs/product.
```

12.7 6) Manual rendering commands (this manual)

```
quarto render docs/user-manual --to html
quarto render docs/user-manual --to pdf
```

12.8 7) Distributed queue (one task)

Full workflow: [Distributed MethylValidation on shared storage](#) (12). One worker invocation:

```
source .venv/bin/activate
methyl-validation run-task --task /work/.../monte_carlo_runs/queue/tasks/run_0001.json
```

12.9 8) Hyperparameter search grid (example)

After `plan-runs` / full stability is understood, use the grid driver. Details: [Optional hyperparameter search](#) (13).

```
source .venv/bin/activate
methyl-hyperparam-search \
  --config /work/experiments/my_mc_config.json \
  --work-dir /work/experiments/hp_run1 \
  --grid '{"stability_dmp_freq": [0.6, 0.7]}' \
  -- --stability
```

12.10 Execution notes

- Always activate `.venv` before `methyl-*` commands.
- Use absolute project paths in production scripts.
- Keep command logs with timestamp and git commit hash for audit.

13 Distributed MethylValidation on shared storage

13.1 Purpose

Run discovery Monte Carlo (and related queue-backed work) on **several machines** when a single host cannot finish `n_iterations` in acceptable wall time. Every worker must read and write the **same** `output_base` tree (commonly shared NFS under `/work/...`).

13.2 Prerequisites

- One **MonteCarloConfig** (dedicated `mc.json` or a project with `step_config.validation`) with `output_base` on shared storage
- The same virtual environment and `methyl-*` CLIs on each worker
- R/W access to: `output_base/<project_name>/monte_carlo_runs/` and the `queue/` subtree created by planning

13.3 Layout (under `monte_carlo_runs/`)

Path	Role
<code>queue/mc_config.json</code>	Snapshotted config for workers
<code>queue/plan_runs.json</code>	Planned runs and task JSON paths
<code>queue/tasks/run_####.json</code>	One task = one <code>run-task</code> invocation
<code>queue/queue_manifest.jsonl</code> / <code>queue/commands.sh</code>	Optional export for a job scheduler
<code>run_####/queue_task_status.json</code>	Worker completion status
<code>run_####/queue_local_step_timings.json</code>	Merged by <code>aggregate-results</code> into <code>step_timings.csv</code>

Full path reference: `packages/methylvalidation/docs/DISTRIBUTED_QUEUE.md`.

13.4 Command sequence

1) **Plan** (one process; creates run dirs and task JSON, no heavy steps):

```
source .venv/bin/activate
methyl-validation plan-runs --config /work/experiments/my_mc_config.json --overwrite
# or: methyl-validation plan-runs --project /work/prostate-cancer/configs/project_Healthy_vs_L
```

`--overwrite` only replaces `queue/tasks/` and regenerates the plan; it does **not** remove existing `run_####` output directories. To delete all `run_####` trees before planning, add `--wipe-runs` (irreversible). Without `--overwrite`, completed runs (see `queue_task_status.json` + per-run task under `queue/tasks/`) are not written over; you can add iterations and only the new `run_####` work is planned. `run-task` skips pipeline work for runs already marked completed unless you pass `--force`.

2) **Export** (optional; for a central queue broker):

```
methyl-validation export-queue --config /work/experiments/my_mc_config.json
```

3) **Workers** (one job per task; repeat until the queue is empty):

```
methyl-validation run-task --task /work/prostate-cancer/Healthy_vs_PCa1-4-CG/monte_carlo_runs/
```

4) **Aggregate** (after successful tasks, or to refresh from a partial set):

```
methyl-validation aggregate-results --config /work/experiments/my_mc_config.json
# add --stability to run the same DMP/gene stability pass as monolithic methyl-validation --st
```

Legacy: a single process may still run `methyl-validation --stability` without subcommands; behavior is unchanged.

13.5 Example paths (/work pattern)

With `output_base` pointing at shared storage, `plan-runs` / `export-queue` resolve `monte_carlo_runs` the same way as the monolithic CLI: `output_base / project_name / monte_carlo_runs /`.

13.6 Recovery

- Re-run a failed `run-task` for the same `--task` path.
- `queue_task_status.json` records failed vs completed.
- `aggregate-results` skips failed runs (when status files are present) and includes runs with computable per-run metrics.

13.7 See also

- `packages/methylvalidation/docs/DISTRIBUTED_QUEUE.md` (canonical detail)
- `docs/theory/chapters/15-model-creation-and-validation.qmd` (section *Distributed Monte Carlo on shared storage*)
- [Optional hyperparameter search](#) (search trials can use the same shared storage)

14 Optional hyperparameter search

14.1 When to use

After a baseline `methyl-validation` path is understood, use a controlled **outer loop** to compare candidate settings—typically **Tier A** knobs in `step_config.validation` / `MonteCarloConfig` (e.g. `stability_dmp_freq`, `n_iterations` as budget, `stability_min_balanced_accuracy`). Tuning detector and model backends (Tier B/C) is a separate, often **nested** exercise; see the theory chapter on multi-stage search tradeoffs in `docs/theory/chapters/15-model-creation-and-validation.qmd` (*Optional pipeline hyperparameter search*).

14.2 What it is

- A **read-only** scalar **objective** (J) is computed from artifacts under `monte_carlo_runs/` (mainly `metrics_summary.json`, optional `stability/stability_summary.json`).
- The `methyl-hyperparam-search` entry point is a small **grid** driver: for each grid point it writes `work_dir/trial_NNNN/mc_config.json` with a merged `MonteCarloConfig` and a distinct `output_base`, runs `methyl-validation` (e.g. with `--stability`), then scores the run.
- It does **not** use `sklearn.model_selection` CV: each evaluation is a subprocess and file read.

Formula and weights: `packages/methylvalidation/docs/HYPERPARAMETER_SEARCH.md` (full (J), `ObjectiveWeights`, constraint semantics).

14.3 Command example

Use a dedicated MC config and a work directory on shared storage if many trials:

```
source .venv/bin/activate
methyl-hyperparam-search \
  --config /work/experiments/my_mc_config.json \
  --work-dir /work/experiments/hp_run1 \
  --grid '{"stability_dmp_freq": [0.6, 0.7], "stability_min_balanced_accuracy": [null, 0.5]}' \
  -- \
  --stability
```

- JSON `null` in `--grid` omits that field so the base config value is kept.
- Results: `work-dir/search_summary.json` (per-trial rows and best feasible trial, if any).
- `--weights-json` can point to a file for `ObjectiveWeights`.

- `--baseline-summary /path/to/metrics_summary.json` enforces **rollout-style** feasibility vs a baseline (same idea as `methyl-validation --rollout-compare`); see `packages/methylvalidation/docs/ROLLOUT.md` and `packages/methylvalidation/docs/HYPERPARAMETER`
- `--dry-run` only writes trial configs and paths; no pipeline subprocesses.

14.4 Pitfalls

- **Discovery vs calibration:** settings that optimize stability panel size may not optimize NLL, Brier, or ECE; prefer a **two-stage** strategy or tight constraints, not a single ad hoc weight mix.
- **n_iterations:** treat primarily as **compute budget** to stabilize medians, not a dimension to grid-search widely without cost awareness.

14.5 See also

- `packages/methylvalidation/docs/HYPERPARAMETER_SEARCH.md`
- `packages/methylvalidation/docs/ROLLOUT.md` (promotion / dual-run thinking)
- `docs/theory/chapters/15-model-creation-and-validation.qmd` (*Optional pipeline hyperparameter search*)
- [Distributed MethylValidation on shared storage](#) (for parallel execution of a single config's iterations)